Session Layer Specification

Mutual Authentication and Secure Messaging
for the NFC Smart Lock Protocol

Version 0.4 – Draft

July 25, 2026

Abstract

This document specifies the cryptographic session module of the NFC smart lock protocol: mutual device authentication and secure-channel establishment between a mobile application ("Phone") and an ESP32-based smart lock controller ("Lock"). It combines long-term Ed25519 identities, ephemeral X25519 key agreement, a three-message SIGMA-style handshake, HKDF-SHA256 key derivation, and AES-256-GCM authenticated encryption to provide mutual authentication, confidentiality, integrity, and forward secrecy for each session. This module has no knowledge of command semantics, authorization policy, or lock actuation – that half of the task belongs to a separate, *equal-standing* module, *smartlock_application_layer.pdf*, with which this document collaborates directly (Section 10). Neither module is a sub-component of the other; each depends on the other for a different half of the same task, and neither is functional alone. This module also makes use of a third, independent component – the transport (*smartlock_transport_layer.pdf*) – to physically move its handshake and ciphertext bytes, but it does not control, configure, or own the transport's state machine or timing; from this module's point of view, the transport is simply an opaque carrier that it hands bytes to and receives bytes from, in the same way the application module hands this module plaintext and receives plaintext back. This revision splits command dispatch, authorization, and actuation out of the previously combined *smartlock_application_layer.pdf* into their own document; see Section 12 for details. No cryptographic behavior changes are introduced by this split.

Contents

1 Scope and Design Assumptions **3**
1.1 Non-Goals . . . 3

2 System Architecture **3**
2.1 Components . . . 3
2.2 Cryptographic Libraries . . . 4

3 Long-Term Identity **4**

4 Handshake Protocol **4**
4.1 Rationale for the Three-Message Design . . . 4
4.2 Message Flow . . . 4
4.3 Message Definitions . . . 5
4.4 Verification Rules . . . 5

5 Shared Secret and Key Derivation **6**
5.1 Shared Secret 6
5.2 Key Derivation 6

6 Secure Communication **6**
6.1 Message Format 6
6.2 Nonce Strategy for Low-Volume Sessions 6
6.3 Replay Protection Within a Session 7

7 Session Termination **7**

8 Security Properties **7**

9 Explicitly Deferred Items (Session-Layer Scope) **8**
9.1 ATECC608A Secure Element Migration 8
9.2 Relay Attack Resistance 9

10 Collaboration with the Application Module **9**

11 Complete Protocol Summary **10**

12 Revision History **11**

1 Scope and Design Assumptions

This specification governs cryptographic session establishment and secure messaging between the Phone and the Lock. The following scope decisions are in effect for this revision and are treated as explicit, reviewed trade-offs rather than oversights.

- **Handshake structure:** Three-message mutual authentication (upgraded from the earlier two-message design) to eliminate signature ambiguity. See Section 4.1.
- **Secure element:** The ESP32 currently stores the long-term Ed25519 private key in software (libsodium, RAM/flash). Migration to an ATECC608A secure element is planned; see Section 9.1.
- **Session message volume:** Each NFC session is expected to carry a low number of application messages (tens, not thousands). This justifies the use of randomly generated AES-GCM nonces without a counter construction; see Section 6.2.
- **Relay attacks:** Out of scope for this revision. This layer does not attempt to bound proximity or handshake latency; see Section 9.2.

1.1 Non-Goals

This specification does not cover: provisioning workflows (manufacturing, QR-code pairing, onboarding UX), firmware update security, physical tamper response, power analysis / side-channel resistance of the current software Ed25519 implementation, transport framing (APDU encapsulation, instruction set, ISO-DEP timing – see *smartlock_transport_layer.pdf*), or anything that happens to plaintext bytes once they are handed to the application module, including command dispatch, authorization policy, and lock actuation (see *smartlock_application_layer.pdf*, a collaborating peer of this document, not a downstream consumer of it). This module authenticates *devices*; it does not authorize *actions* – that determination belongs entirely to the application module.

2 System Architecture

2.1 Components

Component	Platform	Role (this layer)
Mobile Application	Flutter (iOS/Android)	Initiates NFC session, performs the handshake, encrypts/decrypts application payloads. Command content is opaque to this layer – see *smartlock_application_layer.pdf*.
Lock Controller	ESP32 (ESP-IDF)	Verifies Phone identity, performs the handshake, encrypts/decrypts application payloads. Does not interpret plaintext content or actuate hardware – see *smartlock_application_layer.pdf*.

2.2 Cryptographic Libraries

Platform	Library	Responsibility
Flutter	`cryptography`	Ed25519 sign/verify, X25519 agreement, HKDF-SHA256, AES-256-GCM, secure RNG
Flutter	`flutter_secure_storage`	Storage of the Ed25519 private identity key (Keychain / Keystore)
Flutter	`flutter_nfc_kit` / `nfc_manager`	NFC transport only – no cryptographic operations; owned by the transport layer
ESP32	libsodium	Ed25519 sign/verify, X25519 agreement, secure RNG; software key storage today, ATECC608A-backed in a future revision
ESP32	mbedTLS (ESP-IDF)	HKDF-SHA256, AES-256-GCM

No custom cryptographic primitives are implemented on either platform.

3 Long-Term Identity

Each device holds a permanent Ed25519 identity key pair, generated once at provisioning time.

Phone	Ed25519 Private Key (Keychain / Keystore)
	Ed25519 Public Key (provisioned to Lock)
Lock	Ed25519 Private Key (libsodium; ATECC608A in a future revision)
	Ed25519 Public Key (provisioned to Phone / backend)

Public key distribution (manufacturing, QR-code pairing, onboarding) is governed by the trust model and is outside the scope of this document. This specification assumes both parties already hold an authentic copy of the peer's Ed25519 public key before the handshake begins.

4 Handshake Protocol

4.1 Rationale for the Three-Message Design

The original two-message design had the Phone sign "the ephemeral public key and both challenges," but the Phone's first message is sent before it has seen the Lock's challenge. A signature that does not cover the Lock's challenge is not bound to the Lock's contribution to the session, which permits splicing a valid Phone signature from one session into another session carrying a different Lock challenge.

This revision adopts a SIGMA-style three-message handshake: the Lock speaks after the Phone and signs both parties' values, and the Phone closes the handshake with a final signature that also covers both parties' values. Every signature produced by either party covers both ephemeral public keys and both challenges, so no signature is valid outside the session it was created for.

4.2 Message Flow

```
Phone                                                    Lock (ESP32)
-----------------------------------------------------------------------
Generate ephemeral X25519 pair (pk_P^eph, sk_P^eph)
```

```
Generate random challenge c_P

M1: pk_P^eph, c_P  ------------------------------>
                                    Generate ephemeral X25519 pair
                                    (pk_L^eph, sk_L^eph)
                                    Generate random challenge c_L
                                    Sig_L = Sign(pk_P^eph || pk_L^eph
                                                 || c_P || c_L)

               <------------------------------ M2: pk_L^eph, c_L, Sig_L

Verify Sig_L using known PK_L
Sig_P = Sign(pk_P^eph || pk_L^eph || c_P || c_L)

M3: Sig_P  ------------------------------------->
                                    Verify Sig_P using known PK_P

Both sides now hold: authenticated peer identity, authenticated
ephemeral public keys, and two fresh, mutually-bound challenges.
```

4.3 Message Definitions

Msg	Sender	Contents
M1	Phone	pk_P^{eph} (X25519, 32 bytes), c_P (random challenge, 32 bytes)
M2	Lock	pk_L^{eph} (X25519, 32 bytes), c_L (random challenge, 32 bytes), $\text{Sig}_L(pk_P^{eph} \| pk_L^{eph} \| c_P \| c_L)$ (Ed25519, 64 bytes)
M3	Phone	$\text{Sig}_P(pk_P^{eph} \| pk_L^{eph} \| c_P \| c_L)$ (Ed25519, 64 bytes)

Concatenation order $(pk_P^{eph} \| pk_L^{eph} \| c_P \| c_L)$ is fixed and identical for both signatures, and must be implemented identically on both platforms. A protocol version byte and a domain separation prefix (e.g. `"SLOCK-HS-v1"`) SHOULD be prepended to the signed payload on both sides to prevent cross-protocol signature reuse.

4.4 Verification Rules

1. Upon receiving M2, the Phone MUST verify Sig_L using the Lock's provisioned long-term public key PK_L before proceeding to M3. On failure, the Phone MUST abort the session and MUST NOT derive any session key material.

2. Upon receiving M3, the Lock MUST verify Sig_P using the Phone's provisioned long-term public key PK_P before treating the session as authenticated. On failure, the Lock MUST abort; this layer MUST NOT signal a successful handshake to the application layer for this session.

3. Both parties MUST reject a handshake if pk_P^{eph} or pk_L^{eph} is the all-zero point or otherwise fails X25519 validity checks (contributory behavior check / small-subgroup rejection, as provided by libsodium and the `cryptography` package by default).

4. Challenges c_P, c_L MUST be generated with a CSPRNG and MUST NOT be reused across sessions.

5 Shared Secret and Key Derivation

5.1 Shared Secret

Both devices independently compute the X25519 shared secret:

```
SharedSecret = X25519(local_ephemeral_private, remote_ephemeral_public)
```

No key material is transmitted during this step; each side derives the same value from its own ephemeral private key and the peer's ephemeral public key received during the handshake.

5.2 Key Derivation

The session key is derived via HKDF-SHA256, binding in both challenges so that the resulting key is cryptographically tied to this specific handshake instance:

```
PRK = HKDF-Extract(salt = c_P || c_L, ikm = SharedSecret)
Key = HKDF-Expand(PRK, info = "phone->esp" | "esp->phone", L = 32)
```

Two directional keys are derived using distinct info strings to prevent key reuse across communication directions:

Direction	HKDF info string
Phone → Lock	`"phone->esp"`
Lock → Phone	`"esp->phone"`

Each derived key is 32 bytes (256 bits), used directly as an AES-256-GCM key.

6 Secure Communication

6.1 Message Format

All payloads exchanged after the handshake are encrypted with AES-256-GCM using the appropriate directional key. This layer treats the plaintext as an opaque byte string; its structure and interpretation are defined by *smartlock_application_layer.pdf*.

Field	Size	Description
Nonce	12 bytes	CSPRNG-generated, unique per message per key (see Section 6.2)
Ciphertext	variable	AES-256-GCM output for the application-layer plaintext
Auth Tag	16 bytes	GCM authentication tag

6.2 Nonce Strategy for Low-Volume Sessions

Given the low expected number of messages per session (typically a single unlock command and acknowledgment, rarely exceeding a few dozen messages), random 96-bit nonces are considered acceptable for this revision. The birthday-bound collision risk for random GCM nonces becomes significant only as message counts approach the range of 2^{32} under a single key; the per-session ephemeral key lifetime and low message volume keep the collision probability negligible.

Recommendation for future hardening: should session message volume increase substantially (e.g. streaming telemetry, high-frequency polling), this SHOULD be revisited in favor of a deterministic counter-based nonce (optionally salted with a random per-session prefix) to remove reliance on RNG quality for nonce uniqueness.

6.3 Replay Protection Within a Session

Because a new session key is derived per NFC session, and each session is short-lived and single-purpose (e.g. one unlock command), sequence-number based replay protection within a session is currently OPTIONAL. If a session is extended to carry multiple sensitive commands, an incrementing sequence number SHOULD be included in the associated data (AD) passed to AES-GCM. Whether a given plaintext command is safe to replay within a session (e.g. an idempotent status query vs. a state-changing unlock command) is an application-layer policy decision – see *smartlock_application_layer.pdf*.

7 Session Termination

Upon completion or abort of the NFC session, both parties MUST securely erase the following from memory:

- Ephemeral X25519 private key (sk^{eph})
- X25519 shared secret
- HKDF intermediate output (PRK)
- Derived AES-256 session keys (both directions)

Only the long-term Ed25519 identity key persists between sessions. Each subsequent NFC interaction executes the full three-message handshake with freshly generated ephemeral keys and challenges; no session state is resumed or cached. This erase routine is invoked by several trigger conditions defined at the transport layer (session abort, handshake timeout, link loss, frame-integrity error, signature or auth-tag failure); see *smartlock_transport_layer.pdf* §6.

8 Security Properties

Property	Mechanism	Notes
Mutual Device Authentication	Ed25519 signatures over transcript $(pk_P^{eph}, pk_L^{eph}, c_P, c_L)$	Both signatures cover the full transcript, closing the M1-only binding gap of the prior two-message design
Confidentiality	AES-256-GCM	Directional keys via HKDF info separation
Integrity	AES-GCM authentication tag	16-byte tag per message
Key Agreement	X25519	Ephemeral, per-session
Key Derivation	HKDF-SHA256	Challenges bound into salt

Property	Mechanism	Notes
Replay Protection (handshake)	Fresh challenges in signed transcript	Prevents cross-session signature reuse
Forward Secrecy	Fresh X25519 pair per session, destroyed at termination	Compromise of long-term key does not expose past session traffic
Long-Term Key Protection (Phone)	OS Keychain / Keystore via `flutter_secure_storage`	
Long-Term Key Protection (Lock)	Software (libsodium) in this revision; hardware secure element planned	See Section 9.1
Relay Attack Resistance	Not yet implemented – see below	Out of scope this revision; see Section 9.2

Note: device authentication (this table) is a necessary but not sufficient condition for authorizing an action such as unlocking. Whether an authenticated device is *permitted* to issue a given command is defined entirely by *smartlock_application_layer.pdf*.

9 Explicitly Deferred Items (Session-Layer Scope)

The following items are acknowledged gaps, intentionally deferred rather than overlooked, and should be tracked for future revisions of this document. Authorization-model and lock-actuation gaps, previously listed here, now live in *smartlock_application_layer.pdf* (see that document's own deferred-items section); this section retains only cryptographic and handshake-level gaps.

9.1 ATECC608A Secure Element Migration

The ESP32 currently manages its long-term Ed25519 private key in software via libsodium. A future revision will migrate long-term key storage and signing operations to the Microchip ATECC608A secure element, which provides:

- Hardware-isolated private key storage (key never leaves the secure element)
- Hardware-accelerated ECDSA (note: ATECC608A natively supports P-256 ECDSA, not Ed25519 – migration will require either (a) switching the long-term identity scheme to ECDSA P-256, or (b) using the ATECC608A purely as a protected key vault / HMAC-based challenge engine while retaining software Ed25519, depending on final part variant and firmware support)
- Tamper and side-channel resistant signing
- A hardware root of trust for device provisioning

Action item: confirm target ATECC608A variant and its supported curve set before finalizing whether the long-term identity scheme remains Ed25519 or migrates to P-256, as this affects both firmware and the mobile verification code path.

9.2 Relay Attack Resistance

This revision does not defend against relay attacks, in which an attacker relays the NFC exchange between a legitimate Phone and a legitimate Lock over a long-distance channel. Candidate mitigations for future revisions include:

- Distance-bounding / round-trip time (RTT) measurement with a strict timeout on the M1 → M2 → M3 exchange, layered on top of the transport layer's existing handshake timeout T_{HS} (see *smartlock_transport_layer.pdf* §5.1)
- Leveraging NFC's inherent short range together with a hard timeout as a first-order mitigation (does not fully prevent relay but raises attack cost)

10 Collaboration with the Application Module

The session and application modules are peers that jointly implement one task – an authenticated, confidential command exchange – and neither half is meaningful on its own: a Lock that only ran this module could authenticate a Phone and open a secure channel to it, but could never act on anything sent over that channel; a Lock that only ran the application module would have commands to dispatch and hardware to drive, but no way to know whether the byte string requesting them came from a trusted device. Figure 1 shows this as two side-by-side components rather than a stack.

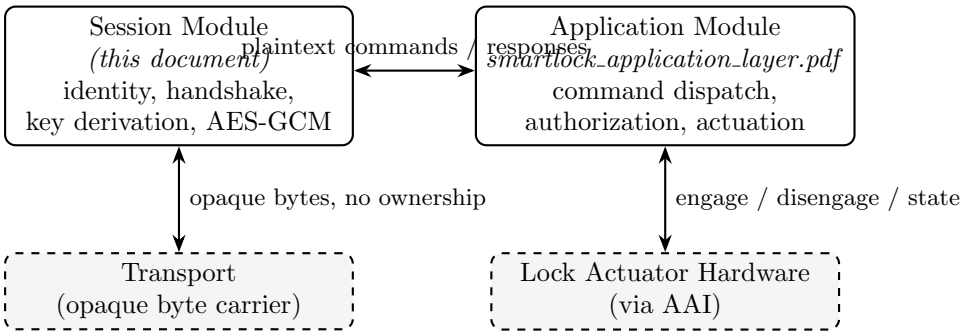

Figure 1: The session and application modules collaborate as peers. Each also has its own, independent external dependency (transport for this module, actuator hardware for the application module) that the other module never touches directly.

- **Session Module → Application Module:** once M3 is verified and the session transitions to *Secure Session*, every successfully decrypted and authenticated plaintext byte string is delivered to the application module unmodified. This module does not parse, validate, or otherwise interpret plaintext content – that is the application module's job, not a job this module delegates from a position above it.
- **Application Module → Session Module:** the application module supplies a plaintext byte string to be encrypted with the appropriate directional key and a fresh nonce; this module returns the nonce, ciphertext, and auth tag for the transport to carry. This module has no visibility into why the application module chose that plaintext.

- **Failure signaling:** an AES-GCM authentication tag mismatch is a failure local to this module and MUST NOT be forwarded to the application module as a plaintext message – it triggers the secure erase routine (Section 10) and session termination directly.
- This module holds no notion of "commands," "authorization," or "actuation." Those are defined entirely in *smartlock_application_layer.pdf*, and this module has no more authority over that document's decisions than it does over the reverse.
- Symmetrically, this module owns the transport relationship shown in Figure 1: the transport (see *smartlock_transport_layer.pdf*) carries this module's bytes without needing to understand them, but this module does not configure, drive, or make decisions on behalf of the transport's own state machine or timing – that remains entirely the transport document's concern.

11 Complete Protocol Summary

```
Phone                                    Lock (ESP32)
-----------------------------------------------------------------
Load Ed25519 identity key                Load Ed25519 identity key
Generate X25519 ephemeral pair (pk_P, sk_P)
Generate challenge c_P

M1: pk_P, c_P  ---------------------------------->
                                  Generate X25519 ephemeral pair
                                  (pk_L, sk_L)
                                  Generate challenge c_L
                                  Sig_L = Sign(pk_P||pk_L||c_P||c_L)

              <---------------------------------- M2: pk_L, c_L, Sig_L

Verify Sig_L using PK_L
Sig_P = Sign(pk_P||pk_L||c_P||c_L)

M3: Sig_P  ------------------------------------->
                                  Verify Sig_P using PK_P

[Both sides] SharedSecret = X25519(local_sk, remote_pk)
[Both sides] PRK    = HKDF-Extract(salt = c_P||c_L, ikm = SharedSecret)
[Both sides] K_p2e = HKDF-Expand(PRK, "phone->esp", 32)
[Both sides] K_e2p = HKDF-Expand(PRK, "esp->phone", 32)

<====== AES-256-GCM encrypted payloads, exchanged with the =======>
<== application module (smartlock_application_layer.pdf) as a peer ==>

[Both sides] Destroy: sk_eph, SharedSecret, PRK, K_p2e, K_e2p
```

12 Revision History

Version	Change	Date
0.1	Initial two-message handshake draft	–
0.2	Upgraded to three-message SIGMA-style handshake to close signature transcript binding gap; documented ATECC608A migration plan; scoped authorization and relay-attack mitigation as deferred work. Published as *smartlock_application_layer.pdf*, combining cryptographic session-establishment content with command dispatch, authorization, and lock-actuation content. (Originally labeled "1.0"; renumbered under 0.3 below to match this document set's sub-1.0 versioning convention.)	July 2, 2026
0.3	**Split out non-cryptographic scope, and renumbered.** Command dispatch, authorization model, and lock actuation logic (formerly §9.2 of the previous revision) moved to a new sibling document, *smartlock_application_layer.pdf* (v0.1). This document renamed to *smartlock_session_layer.pdf* and its scope narrowed to cryptographic session establishment and secure messaging only, aligning with the component map already published in *smartlock_transport_layer.pdf* §1.2. Added a data-contract section formalizing the interface with the new application layer document. Version numbering corrected from "1.1" to follow the project-wide convention that no document in this set reaches 1.0 before a stable freeze (see the numbering note below). No cryptographic behavior changes.	July 24, 2026
0.4	**Reframed as a peer module, not a stacked layer.** Former "Interface to the Application Layer" section (with "upward"/ "downward" language) replaced by §10, "Collaboration with the Application Module," which presents the session and application modules as equal-standing collaborators (Figure 1) rather than one sitting on top of the other. Clarified throughout that this module does not own, configure, or control the transport's state machine – the transport is an independent, opaque byte carrier used by this module, exactly as this module is used by the application module. No cryptographic or behavioral changes.	July 25, 2026

Note on numbering: this project has not yet reached a stable v1.0 release. Version numbers below 1.0 are used throughout the document set (session, application, transport, hardware/link layer, LLI) until the first frozen release, and are not directly comparable across documents – e.g. this document's

0.4 does not imply it is "ahead of" or "behind" application layer 0.2 in maturity, only that it is this document's fourth drafted revision. An earlier draft of this document briefly used a 1.x numbering scheme (1.0–1.2); that was inconsistent with the convention followed by the rest of the document set and has been corrected above.